

Week 3: Seller and Inventory Management System

Scenario:

The e-commerce company sells products supplied by multiple sellers. The system should maintain seller details, track products supplied by each seller, and monitor inventory levels to ensure product availability.

1. Create Seller and Inventory Tables:

```
SQL> CREATE TABLE Seller(Seller_ID VARCHAR(7),Seller_Name VARCHAR(25),Email VARCHAR(30),Phone NUMBER(10),Address VARCHAR(45));
Table created.

SQL> CREATE TABLE Inventory(Inventory_ID VARCHAR(7),Product_ID VARCHAR(7),Seller_ID VARCHAR(7),Stock_Quantity NUMBER,Stock_Status VARCHAR(11),Last_Updated DATE);
Table created.
```

Shows the CREATE TABLE commands used to create the Seller and Inventory tables.

2. Establish Relationships:

Seller and Product:

One seller can supply multiple products.

One product can be supplied by one or multiple sellers.

Relationship:

Seller → Product is one to many

Product and Inventory:

Each product has inventory information.

Inventory tracks available stock.

Relationship:

Product → Inventory is one to one

Primary Key added for Both Tables:

Shows the primary key constraints added to the Category and Product tables.

```
SQL> ALTER TABLE Seller ADD CONSTRAINT PK_Seller PRIMARY KEY(Seller_ID);
Table altered.

SQL> ALTER TABLE Inventory ADD CONSTRAINT PK_Inv PRIMARY KEY(Inventory_ID);
Table altered.
```

Foreign Key for Inventory Table:

Shows the foreign key constraint added to the Inventory table.

```
SQL> ALTER TABLE Inventory ADD CONSTRAINT FK_Prod FOREIGN KEY(Product_ID) REFERENCES Product(Product_ID);  
Table altered.
```

```
SQL> ALTER TABLE Inventory ADD CONSTRAINT FK_Seller FOREIGN KEY(Seller_ID) REFERENCES Seller(Seller_ID);  
Table altered.
```

Constraints to make Unique Email ID:

Shows the unique constraint added to the Seller email column.

```
SQL> ALTER TABLE Seller ADD CONSTRAINT Un_mail UNIQUE(Email);  
Table altered.
```

To make Email NOT NULL:

Shows the NOT NULL constraint applied to the Seller email column.

```
SQL> ALTER TABLE Seller MODIFY Email NOT NULL;  
Table altered.
```

Add Constraint to Check the Stock Quantity is Positive:

Shows the CHECK constraint ensuring Stock_Quantity is greater than zero.

```
SQL> ALTER TABLE Inventory ADD CONSTRAINT CK_Quan CHECK(Stock_Quantity>=0);  
Table altered.
```

Insert Values to Seller Table:

Shows seller records being inserted into the Seller table.

```
SQL> insert into seller values('S1001','ABC Electronics','abc@gmail.com',9876543210,'Chennai');  
1 row created.  
SQL> insert into seller values('S1002','Fashion hub','hub@gmail.com',9876543211,'Tambaram');  
1 row created.  
SQL> insert into seller values('S1003','Daily needs','daily@gmail.com',9876543212,'Chrompet');  
1 row created.  
SQL> insert into seller values('S1004','Home Furniture','furniture@gmail.com',9876543205,'Velachery');  
1 row created.  
SQL> insert into seller values('S1005','Book World','books@gmail.com',9876543218,'T nagar');  
1 row created.
```

Shows the Seller table after inserting and displaying the seller records.

```
SQL> select*from seller;
```

SELLER_	SELLER_NAME	EMAIL	PHONE	ADDRESS
S1001	ABC Electronics	abc@gmail.com	9876543210	Chennai
S1002	Fashion hub	hub@gmail.com	9876543211	Tambaram
S1003	Daily needs	daily@gmail.com	9876543212	Chrompet
S1004	Home Furniture	furniture@gmail.com	9876543205	Velachery
S1005	Book World	books@gmail.com	9876543218	T nagar

Insert Values to Inventory Table:

Shows inventory records being inserted into the Inventory table.

```
SQL> insert into inventory values('I1001','P1002','S1001',50,'In stock',sysdate);
```

```
1 row created.
```

```
SQL> insert into inventory values('I1002','P1003','S1002',40,'In stock',sysdate);
```

```
1 row created.
```

```
SQL> insert into inventory values('I1003','P1004','S1004',15,'In stock',sysdate);
```

```
1 row created.
```

```
SQL> insert into inventory values('I1004','P1005','S1005',25,'In stock',sysdate);
```

```
1 row created.
```

```
SQL> insert into inventory values('I1005','P1006','S1001',30,'In stock',sysdate);
```

```
1 row created.
```

Shows the Inventory table records after insertion.

```
SQL> select*from inventory;
```

INVENTO	PRODUCT	SELLER_	STOCK_QUANTITY	STOCK_STATU	LAST_UPDA
I1001	P1002	S1001	50	In stock	20-SEP-26
I1002	P1003	S1002	40	In stock	20-SEP-26
I1003	P1004	S1004	15	In stock	20-SEP-26
I1004	P1005	S1005	25	In stock	20-SEP-26
I1005	P1006	S1001	30	In stock	20-SEP-26

Report 1: Seller-wise Product Report

Display:

- Seller name
- Number of products supplied
- Available stock

Shows the Seller-wise Product Report with seller name, product count, and available stock.

```
SQL> SELECT s.Seller_Name,  
2 COUNT(i.Product_ID) AS Number_of_Products,  
3 SUM(i.Stock_Quantity) AS Available_Stock  
4 FROM Seller s  
5 JOIN Inventory i ON s.Seller_ID = i.Seller_ID  
6 WHERE i.Stock_Status = 'In stock'  
7 GROUP BY s.Seller_Name;
```

SELLER_NAME	NUMBER_OF_PRODUCTS	AVAILABLE_STOCK
Fashion hub	1	40
Home Furniture	1	15
Book World	1	25
ABC Electronics	2	80

Use Joins to get details from Seller and Inventory Table.

Report 2: Stock Availability Report

Display:

- Product name
- Stock quantity
- Stock status

Shows the Stock Availability Report with product name, stock quantity, and stock status.

```
SQL> SELECT p.Product_Name, i.Stock_Quantity, i.Stock_Status
2 FROM Product p
3 JOIN Inventory i ON p.Product_ID = i.Product_ID;

PRODUCT_NAME
-----
STOCK_QUANTITY STOCK_STATU
-----
T-shirt        50 In stock
Rice bag       40 In stock
Office chair   15 In stock
Python book    25 In stock
Headphones     30 In stock
```

Report 3: Inventory Analysis:

Find:

- Total products available

Shows the total number of products currently available.

```
SQL> SELECT COUNT(*) AS Total_Available_Products
2 FROM Inventory
3 WHERE Stock_Status = 'In stock';

TOTAL_AVAILABLE_PRODUCTS
-----
5
```

• Products out of stock:

Shows the number of products that are out of stock.

```
SQL> SELECT COUNT(*) AS OutOfStock_Products
2 FROM Inventory
3 WHERE Stock_Status = 'Out of Stock';

OUTOFSTOCK_PRODUCTS
-----
0
```

• Highest stocked products:

Shows the product(s) with the highest stock quantity.

```
SQL> SELECT p.Product_Name, i.Stock_Quantity
2  FROM Product p
3  JOIN Inventory i ON p.Product_ID = i.Product_ID
4  WHERE i.Stock_Quantity = (SELECT MAX(Stock_Quantity) FROM Inventory);
```

```
PRODUCT_NAME
```

```
STOCK_QUANTITY
```

```
T-shirt          50
```

- **Average inventory quantity:**

Shows the average inventory quantity calculated using AVG(Stock_Quantity).

```
SQL> SELECT AVG(Stock_Quantity) AS Average_Inventory
2  FROM Inventory;
```

```
AVERAGE_INVENTORY
```

```
32
```